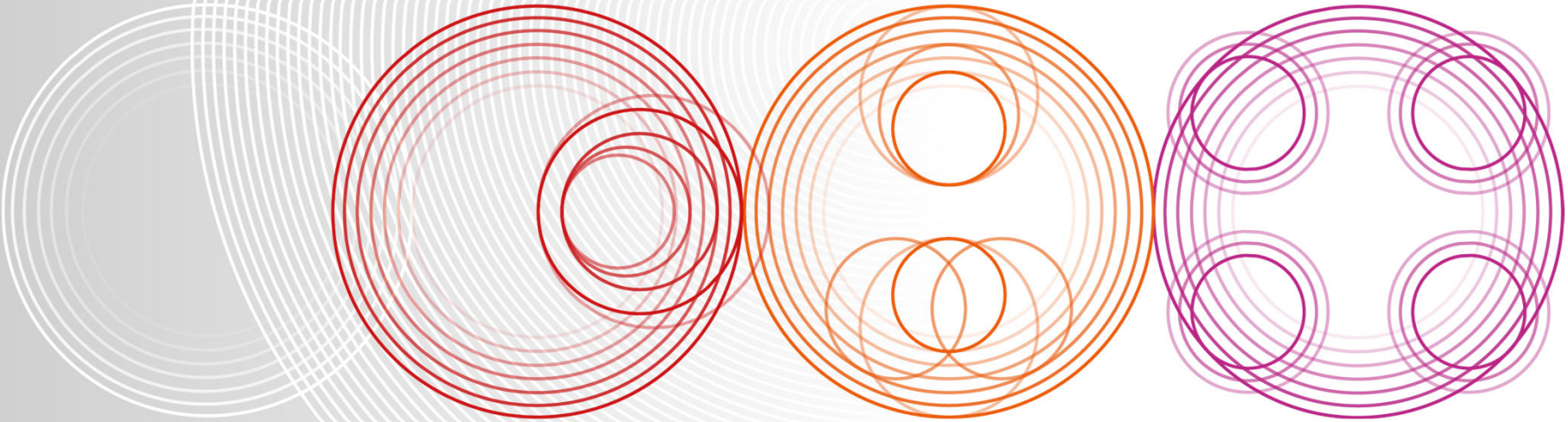
2026-09-30

Coding und Digitalisierung



Organisation

Vorlesung immer Donnerstags von 9:00 Uhr bis 12:30.

Coding und Digitalisierung ist ein Langläufer Modul, d.h. es läuft über das gesamte Semester.

Aufgeteilt in zwei Blöcke.

1. Block: VL und vermittlung der theoretischen Grundlagen (Peter Kern)
2. Block: Projektarbeit (Mirko Lotz-Blumberg)

Block 1:

- Insgesamt 7 Veranstaltungen + Übungen im Selbststudium.
- Schriftliche Klausur von 90 Minuten

Die Noten aus Block 1 und Block 2 werden jeweils mit 50% gewichtet und ergeben die Gesamtnote.

Sie müssen beide Veranstaltungen bestehen.

Organisation

Anerkennung von Prüfungsleistungen

Alte Prüfungsleistung	Anrechnung in Coding und Digitalisierung
Projekt	Projekt
Klausur Informatik 1	Klausur Teil A (Grundlagen)
Klausur Informatik 2	Klausur Teil B (erweiterte Techniken)

Ergänzend werden die Klausuren in den bisherigen Fächern „Informatik 1“ und „In-formatik 2“ noch ein letztes Mal angeboten werden.

Python

- **Die Sprache:** Eine der weltweit beliebtesten, vielseitigsten Programmiersprachen (High-Level).
- **Der Fokus:** Extrem leicht lesbarer und strukturierter Code (Fokus auf Einfachheit).
- **Der Schöpfer:** Entwickelt von dem Niederländer Guido van Rossum.
- **Das Jahr:** Erste Veröffentlichung im Jahr 1991.
- **Der Name:** Inspiriert von der britischen Komikertruppe *Monty Python's Flying Circus* – nicht von der Würgeschlange!



Wie funktioniert Python

Obwohl Python oft als reine "interpretierte Sprache" bezeichnet wird, arbeitet sie unter der Haube in zwei Schritten:

- **Kompilierung in Bytecode:** Wenn du dein Programm (die .py-Datei) ausführst, übersetzt Python den menschenlesbaren Quellcode im Hintergrund zuerst in diesen plattformunabhängigen Bytecode.
- **Ausführung durch die Virtual Machine:** Dieser Bytecode wird dann an die Python Virtual Machine (PVM) übergeben. Die PVM ist das eigentliche Herzstück (der Interpreter), das den Bytecode liest, Anweisung für Anweisung übersetzt und auf dem Prozessor deines jeweiligen Betriebssystems ausführt.



Funktionsweise der Python Virtual Machine (PVM)

Grundkonzept

- Die PVM ist keine eigenständige Virtualisierungssoftware (wie VMWare), sondern das Software-Herzstück des Python-Interpreters (die Laufzeitumgebung).
- Sie fungiert als kontinuierliche Schleife, die den Code schrittweise verarbeitet.

Der Ausführungsprozess

- Einlesen (Fetch): Der zuvor erstellte plattformunabhängige Bytecode wird Anweisung für Anweisung eingelesen.
- Dekodieren & Ausführen: Die PVM agiert als Dolmetscher. Jeder Bytecode-Befehl löst in der PVM vorbereitete, maschinennahe Routinen (bei der Standard-Version in C) aus, die vom Prozessor verarbeitet werden.

Grundlegende Eigenschaften von Python

Syntax und Struktur

- **Einrückung (Indentation):** Im Gegensatz zu Sprachen wie Java oder C verwendet Python keine geschweiften Klammern {} für Code-Blöcke. Die Struktur wird zwingend durch Einrückung (meist 4 Leerzeichen) definiert.
- **Keine Semikolons:** Zeilenenden markieren das Ende eines Befehls; Semikolons sind nicht erforderlich.
- **Kommentare:** Werden mit einer Raute # eingeleitet und vom Interpreter ignoriert.

```
if wahr:
    print("Dieser Block ist eingerückt")
```

Grundlegende Eigenschaften von Python

Variablen und dynamische Typisierung

- **Dynamische Typisierung:** Der Datentyp einer Variablen wird automatisch zur Laufzeit zugewiesen. Es ist keine explizite Deklaration (wie int x = 5;) nötig.

- **Grundlegende Datentypen:**
  - int: Ganzzahlen (z. B. 42)
  - float: Gleitkommazahlen (z. B. 3.14)
  - str: Zeichenketten/Strings (z. B. "Hallo")
  - bool: Wahrheitswerte (True oder False)

```
messwert = 23.5 # Python erkennt automatisch 'float
```

Grundlegende Eigenschaften von Python

Operatoren

- **Arithmetische Operatoren:** + (Addition), - (Subtraktion), * (Multiplikation), / (Division)
- **Spezielle Arithmetik:** // (Ganzzahldivision), % (Modulo/Rest), ** (Potenzierung)
- **Vergleichsoperatoren:** == (gleich), != (ungleich), <, >, <=, >=
- **Logische Operatoren:** and, or, not für die Verknüpfung von Wahrheitswerten

```
if wahr:
    print("Dieser Block ist eingerückt")
```

Grundlegende Eigenschaften von Python

Kontrollstrukturen – Verzweigungen

- **Zweck:** Bedingte Ausführung von Code-Blöcken basierend auf Wahrheitswerten.
- **Schlüsselwörter:** if, elif (else if), else.

```
temperatur = 25

if temperatur > 30:
    print("Kühlung aktivieren")
elif temperatur < 10:
    print("Heizung aktivieren")
else:
    print("Temperatur im Normalbereich")
```

Grundlegende Eigenschaften von Python

Kontrollstrukturen – Schleifen (Loops)

- **for-Schleife:** Iteriert über eine Sequenz (wie Listen, Strings oder Zahlenreihen).
  - Hilfsfunktion: range(start, stop, step) erzeugt Zahlenfolgen.
- **while-Schleife:** Führt Code aus, solange eine bestimmte Bedingung True ist.
- **Schleifensteuerung:**
  - break: Bricht die Schleife sofort ab.
  - continue: Überspringt den restlichen Code im aktuellen Durchlauf und macht beim nächsten weiter.

Grundlegende Eigenschaften von Python

Datenstrukturen – Listen und Tupel

- **Listen (list):** Geordnete, veränderbare (mutable) Sammlungen von Elementen. Definiert mit eckigen Klammern [].
  - **Methoden: .append(), .remove(), .sort()**

- **Tupel (tuple):** Geordnete, *unveränderbare* (immutable) Sammlungen. Definiert mit runden Klammern (). Sicherer für konstante Datensätze.

- **Indexierung:** Der Zugriff erfolgt in beiden Fällen über den Index, beginnend bei 0 (z. B. meine_liste[0])

Grundlegende Eigenschaften von Python

Datenstrukturen – Dictionaries

- **Konzept:** Speichern von Daten in Schlüssel-Wert-Paaren (Key-Value). Ähnlich einem echten Wörterbuch oder einer Hashmap.
- **Eigenschaften:** Schlüssel müssen eindeutig und unveränderbar (z. B. Strings, Zahlen) sein. Definiert mit geschweiften Klammern {}.
- **Zugriff:** Sehr schneller Zugriff auf Werte über ihren Schlüssel.

```
sensor_daten = {"id": 101, "typ": "Temperatur", "wert": 22.4}
print(sensor_daten["wert"]) # Gibt 22.4 aus
```

Grundlegende Eigenschaften von Python

Module und Bibliotheken (Import-System)

- **Modularisierung:** Auslagerung von Code in separate Dateien (.py), um große Projekte zu strukturieren.
- **Standardbibliothek:** Python bringt "Batteries Included" – viele integrierte Module für Mathematik, Datum/Zeit, Dateisysteme etc.

```
import math
wurzel = math.sqrt(16)

from datetime import datetime
jetzt = datetime.now()
```

Grundlegende Eigenschaften von Python

Operatoren

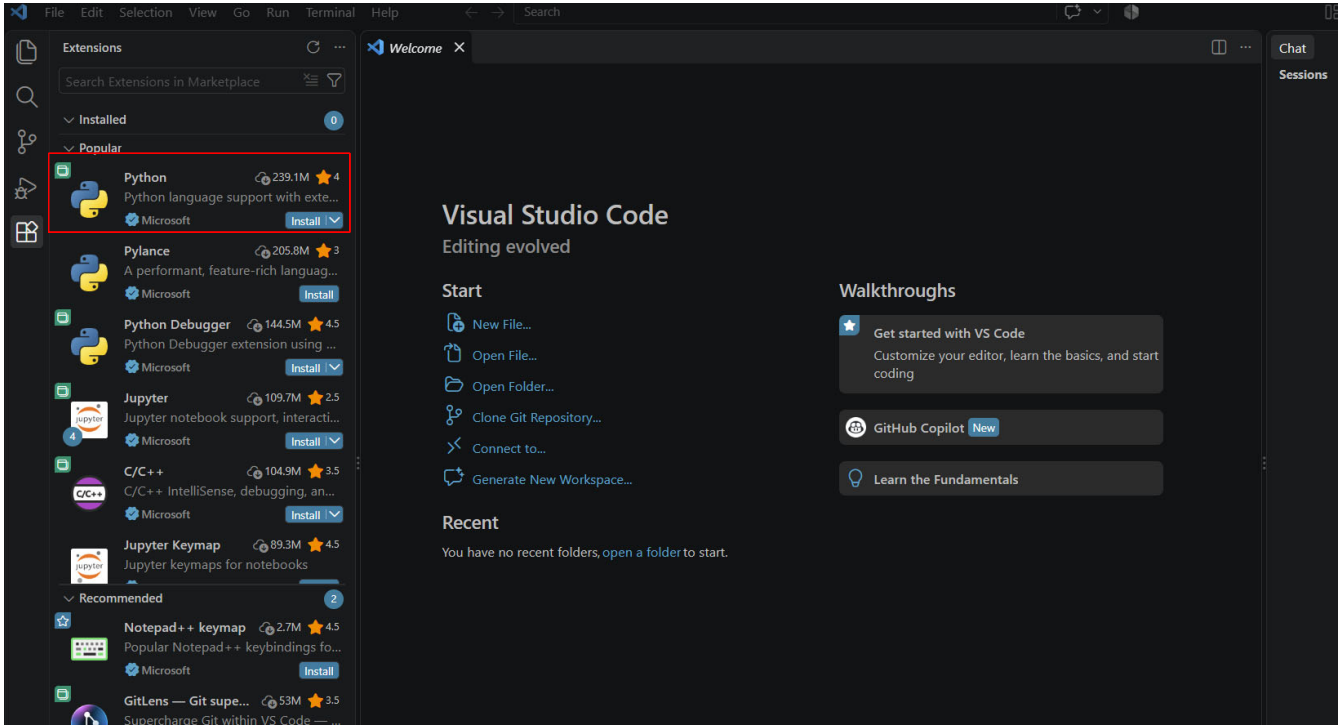
- **Arithmetische Operatoren:** + (Addition), - (Subtraktion), * (Multiplikation), / (Division)
- **Spezielle Arithmetik:** // (Ganzzahldivision), % (Modulo/Rest), ** (Potenzierung)
- **Vergleichsoperatoren:** == (gleich), != (ungleich), <, >, <=, >=
- **Logische Operatoren:** and, or, not für die Verknüpfung von Wahrheitswerten

```
if wahr:
    print("Dieser Block ist eingerückt")
```

Installation

Voraussetzungen installieren: Python und VS Code vorbereiten.

Stelle sicher, dass **Python** auf deinem Computer installiert ist. Öffne dann VS Code, wechsle in die Ansicht für Erweiterungen (Strg+Umschalt+X oder Cmd+Umschalt+X auf dem Mac) und installiere die offizielle **Python-Erweiterung** von Microsoft.



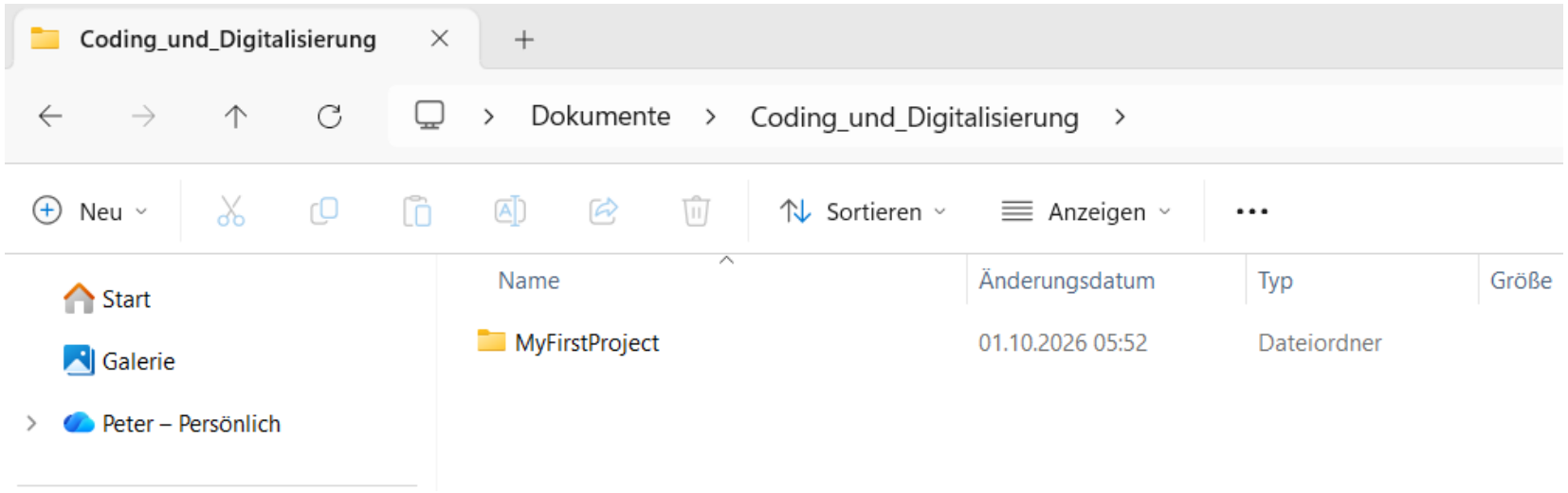
https://www.python.org/

https://code.visualstudio.com/

Installation

Projektordner erstellen und öffnen:

Erstelle auf deinem Computer einen neuen, leeren Ordner für dein Projekt (z.B. MeinPythonProjekt). Ziehe diesen Ordner entweder per Drag-and-Drop in VS Code oder öffne ihn über das Menü **Datei > Ordner öffnen...** (File > Open Folder...).



Installation

Virtuelle Umgebung (venv) erstellen: Isoliert Pakete von anderen Projekten.

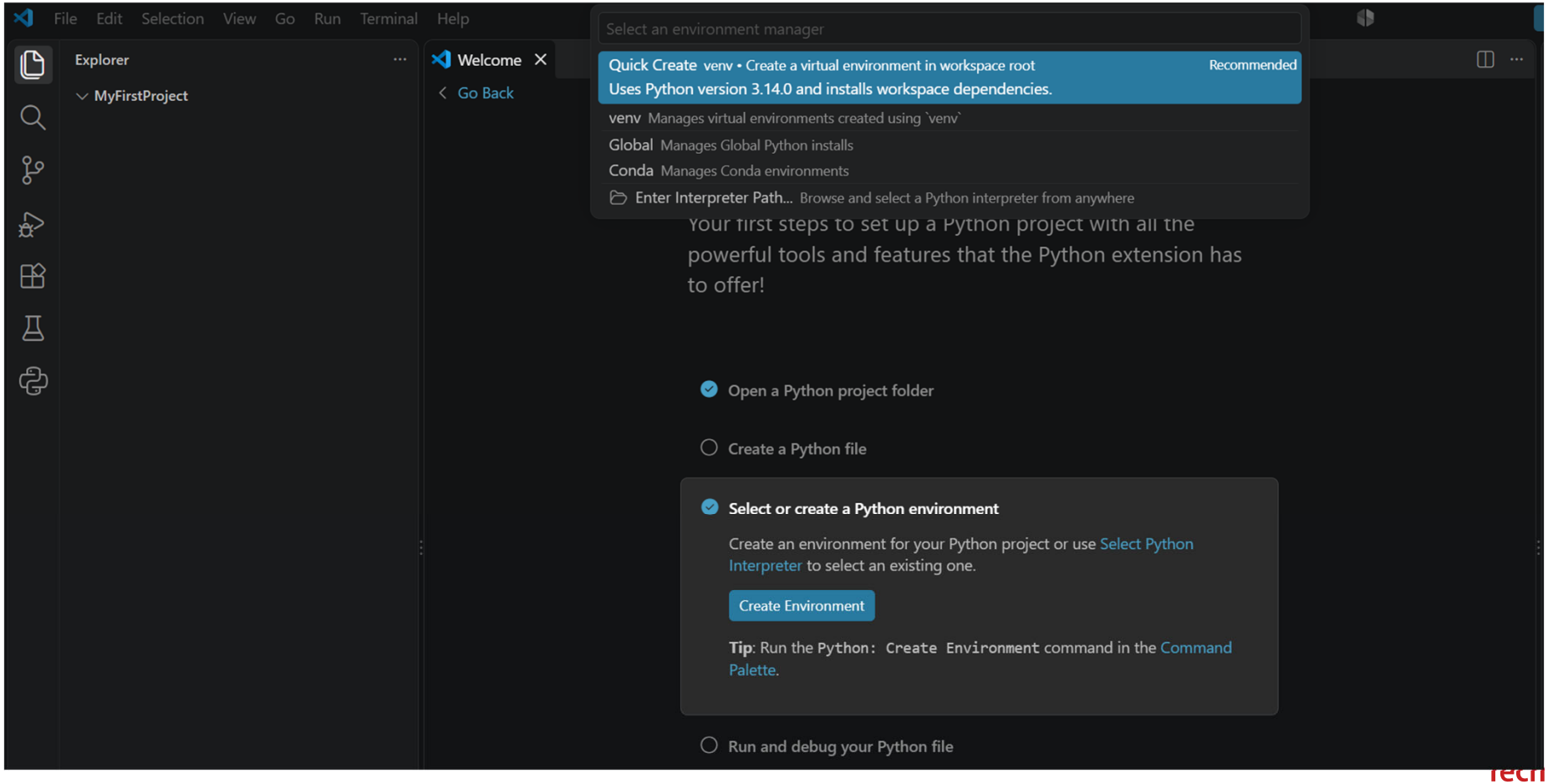
Öffne das integrierte Terminal in VS Code über **Terminal > Neues Terminal** (Strg+ö oder Ctrl+`). Gib folgenden Befehl ein und drücke Enter:

Windows: python -m venv .venv

Mac/Linux: python3 -m venv .venv

VS Code fragt dich unten rechts oft direkt, ob diese neue Umgebung für den Arbeitsbereich verwendet werden soll. Klicke auf **Ja**.

Installation



Erster Test

5.Erste Datei erstellen und ausführen:

Klicke im Datei-Explorer am linken Rand auf das Symbol "Neue Datei" und nenne sie main.py. Füge einen Test-Code ein:

```
print("Hallo! Willkommen im Kurs Coding und Digitalisierung.")
name = input("Wie lautet dein Name? ")
print(f"Freut mich, dich kennenzulernen, {name}!")
```

Vielen Dank

